



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS III: Ingeniería de los sistemas de información

**Tema 96. Procesos de pruebas y garantía de calidad en el
desarrollo de software. Planificación, estrategia de pruebas y
estándares. Niveles, técnicas y herramientas de pruebas de
software. Criterios de aceptación de software**

AUTOR: Jesús Lozano Robles

Actualización 2017

ÍNDICE

1	PROCESOS DE PRUEBAS Y GARANTÍA DE CALIDAD EN EL DESARROLLO SOFTWARE.....	4
1.1	QUÉ ES CALIDAD.....	4
1.2	MOTIVOS PARA CENTRARSE EN LA CALIDAD EN EL SOFTWARE	6
1.3	ASEGURAMIENTO DE CALIDAD VS CONTROL DE CALIDAD	7
1.4	ESTÁNDARES DE TESTING.....	8
1.4.1	ESTÁNDARES DE TESTING DE “IURE”	8
1.4.2	ESTÁNDARES DE TESTING DE “FACTO”	9
1.4.3	PRINCIPIOS DE LAS PRUEBAS SOFTWARE	10
1.5	CONCLUSIONES	10
2	PLANIFICACIÓN, ESTRATEGIA Y GESTIÓN DE LAS PRUEBAS.....	12
2.1	CICLO DE VIDA GENÉRICO	12
2.2	GESTIÓN DE LAS PRUEBAS.....	13
2.2.1	ROLES.....	13
2.2.2	PLANIFICACIÓN	14
2.2.3	ESTIMACIÓN DE LAS PRUEBAS.	15
2.2.4	ESTRATEGIA DE PRUEBAS	15
2.2.5	RIESGOS.....	16
2.3	CICLO DE VIDA DE EJECUCIÓN DE LAS PRUEBAS.	17
2.4	CONCLUSIONES.	18
3	CLASIFICACIÓN Y TIPOS DE PRUEBAS	19
3.1	TÉCNICAS ESTÁTICAS.....	20
3.1.1	REVISIONES	20
3.1.1.1	ROLES EN LAS REVISIONES	20
3.1.1.2	FASES DE LA REVISIÓN	21
3.1.1.3	TIPOS DE REVISIÓN	21
3.1.2	ANÁLISIS ESTÁTICO O DICTÁMENES APOYADOS POR HERRAMIENTAS.....	22
3.1.2.1	COMPLEJIDAD CICLOMÁTICA.....	22
3.1.2.2	CÓDIGO DUPLICADO	22
3.1.2.3	CICLOS ENTRE PAQUETES	22
3.2	TÉCNICAS DINÁMICAS O TÉCNICAS DE DISEÑO DE PRUEBAS.	23
3.2.1	PRUEBAS DE CAJA BLANCA	24
3.2.1.1	COBERTURA DE SENTENCIAS	24
3.2.1.2	COBERTURA DE RAMAS	24
3.2.1.3	COBERTURA DE CONDICIONES.....	25
3.2.1.4	COBERTURA DE RUTAS.....	25
3.2.2	PRUEBAS CAJA NEGRA.....	25
3.2.2.1	CLASES DE EQUIVALENCIA	25

3.2.2.2	ANÁLISIS DE VALORES LÍMITE	26
3.2.2.3	TABLA DE DECISIÓN.....	26
3.2.2.4	PRUEBAS DE ESTADO.....	26
3.2.2.5	PRUEBAS DE CASOS DE USO	26
3.3	CRITERIOS DE ACEPTACIÓN DEL SOFTWARE	27
3.4	CONCLUSIONES	27
4	PROCESOS DE PRUEBAS EN METODOLOGÍAS ÁGILES..	29
4.1	FACTORES DE ÉXITO PARA LA IMPLANTACIÓN DE PRUEBAS ÁGILES.	29
4.2	EL PAPEL DEL RESPONSABLE DE PRUEBAS EN EQUIPOS ÁGILES.....	30
4.3	FRAMEWORKS ÁGILES.	30
4.3.1	TDD	30
4.3.2	ATDD	31
4.3.3	BDD	31
4.4	CONCLUSIONES	33
5	HERRAMIENTAS DE PRUEBAS	34
5.1	JUSTIFICACIÓN A LA NECESIDAD DE LA AUTOMATIZACIÓN DE PRUEBAS.....	34
5.2	HERRAMIENTAS DE DISEÑO Y GESTIÓN DE PRUEBAS.....	34
5.3	HERRAMIENTAS DE AUTOMATIZACIÓN DE PRUEBAS Y MÉTRICAS DE CALIDAD ..	34
5.4	CONCLUSIONES	35
6	GLOSARIO	37

1 Procesos de pruebas y garantía de calidad en el desarrollo software.

En este tema vamos a tratar el concepto de calidad del software, a los procesos habituales y las técnicas que se suelen utilizar para probar software, tanto desde el punto de vista de las pruebas clásicas, como el hecho de conseguir métricas que nos digan la calidad del código. Se reservará un apartado para las metodologías ágiles. Finalmente se terminará con las herramientas clásicas en materia de gestión de pruebas y su automatización.

Para empezar, en este apartado, se comenzará por el concepto de calidad, la justificación de por qué se ha de tener procesos de calidad, la diferencia entre aseguramiento de calidad y control de calidad y los principales estándares existentes.

1.1 Qué es calidad.

El término calidad es bastante ambiguo. En función del ámbito, puede significar una cosa u otra. Por ejemplo, la calidad del software en los programas que controlan los aviones se relaciona con el seguimiento de la normativa aeronáutica.

Sin embargo, calidad en un producto como el buscador Google, o el correo Outlook está más relacionado con lo fácil que pueda ser para el usuario el uso de la aplicación y la funcionalidad que provee, ya que no existe una definición formal de correcto como puede ser el caso de los aviones.

Debido a esto, el concepto de calidad ha de incorporar al menos los siguientes elementos:

- Ausencia de fallos (o al menos de fallos importantes).
- Satisfacción del usuario (cosa difícil de medir de manera objetiva y comparable).
- Conformidad con los requerimientos.

La norma técnica ISO/EIC 25010:2011 define la calidad como un conjunto de atributos que ha de tener el software, que son desglosados en los siguientes:

1. **Adecuación Funcional:** Que se define como la capacidad de un producto software de satisfacer las necesidades declaradas e implícitas, pudiéndose subdividir de la siguiente forma:
 - a. **Corrección:** Que el software desarrolla lo deseado por el cliente de manera precisa.
 - b. **Compleitud:** Que desarrolla toda la funcionalidad requerida.
 - c. **Pertinencia:** Que la funcionalidad desarrollada sea pertinente.

Por ejemplo, se pueden tener todos los elementos típicos de un gestor de correo, a saber: recepción y envío de correos, pero sin embargo puede haber casos en los cuales el correo no se envíe o que, en función del remitente, el correo llegue en una codificación rara. Este sería un software completo, puesto que tiene todas las funcionalidades, pero no sería del todo correcto, puesto que contiene errores algunas de las funcionalidades.

2. **Portabilidad:** Se define como la facilidad de cambiar el entorno del sistema, ya sea software, hardware u operacional. Cuanto más portable y menos dependiente sea de una tecnología, más calidad tendrá el sistema. Tiene las siguientes subcaracterísticas:
 - a. **Adaptabilidad:** Define la capacidad de adaptarlo a los nuevos entornos.

- b. **Capacidad de ser instalado:** Define la facilidad para ser instalado y desinstalado con éxito.
 - c. **Capacidad de ser reemplazado:** capacidad para ser sustituido por otro software con el mismo propósito y en el mismo entorno.
- 3. **Fiabilidad:** Es la capacidad que tiene un sistema o componente, según la funcionalidad requerida, bajo unas condiciones de carga de trabajo definidas o el comportamiento esperado ante fallos del sistema.
 - a. **Madurez:** Se define como la capacidad del sistema de satisfacer las necesidades en condiciones normales de uso.
 - b. **Disponibilidad:** Se define como la capacidad de un sistema de estar operativo y en funcionamiento cuando se requiere.
 - c. **Tolerancia a fallos:** Se define como la capacidad del sistema o componente para funcionar según lo previsto ante fallos software o hardware.
 - d. **Capacidad de recuperación:** Es la propiedad de un producto software de recuperar los datos y el estado previos a la interrupción o fallo del mismo.
- 4. **Usabilidad:** Capacidad de un producto para ser entendido, aprendido, usado y resultar atractivo para el usuario, cuando se usa bajo determinadas condiciones. Sus subcaracterísticas son:
 - a. **Capacidad para reconocer su adecuación:** Se puede definir como la capacidad del producto que permite al usuario ver si el software es adecuado a sus necesidades.
 - b. **Capacidad de aprendizaje:** Propiedad del producto que permite al usuario aprender su manejo.
 - c. **Capacidad para ser usado:** Propiedad del producto que permite al usuario controlarlo con facilidad.
 - d. **Protección contra errores de usuario:** Capacidad del sistema de proteger al usuario de sus propios errores.
 - e. **Estética de la interfaz de usuario:** Propiedad del interfaz de usuario de ser agradable y atractivo en su uso.
 - f. **Accesibilidad:** Capacidad del producto que permite que sea utilizado por otros usuarios con determinadas discapacidades. Se hablará de accesibilidad en otro apartado del temario de la oposición.
- 5. **Eficacia en el desempeño:** Que se refiere al uso de los recursos de la manera más reducida posible. Permite la escalabilidad de las aplicaciones y el uso óptimo de los recursos hardware de la organización. Se subdivide en las siguientes partes:
 - a. **Comportamiento temporal:** Aquella propiedad en la que los tiempos de respuesta y los ratios de productividad tienen un buen comportamiento comparado con respecto de su benchmark.
 - b. **Utilización de recursos:** Se define como la cantidad y tipo de recursos utilizados por el software.
 - c. **Capacidad:** Nivel máximo en el que los parámetros, de un software o un sistema, puede funcionar de manera apropiada.
- 6. **Compatibilidad:** Capacidad de dos o más sistemas o componentes para intercambiar información y/o llevar a cabo sus funciones requeridas cuando comparten el mismo entorno hardware o software. Sus principales partes son:
 - a. **Coexistencia:** Propiedad mediante la cual dos o más elementos pueden compartir recursos en un entorno común.
 - b. **Interoperabilidad:** Propiedad que permite el intercambio de información entre varios sistemas.

7. **Mantenibilidad:** Se define como la facilidad para realizar modificaciones en el software. Cuanto mayor sea el esfuerzo, habitualmente medido en horas/persona, menor será la mantenibilidad del mismo. Sus subcaracterísticas son:
- a. **Modularidad:** Capacidad para que un componente de un sistema o programa se pueda modificar con un impacto mínimo.
 - b. **Reusabilidad:** Capacidad de un componente de un sistema o programa para que sea reutilizado en otros sistemas o programas.
 - c. **Analizabilidad:** Facilidad con la que se puede evaluar el impacto de un determinado cambio o de evaluación de posibles problemas en el código.
 - d. **Capacidad de ser modificado:** capacidad para la modificación de un producto sin introducir defectos o degradar su funcionamiento.
 - e. **Capacidad de ser probado:** Facilidad con la que se pueden establecer criterios de prueba y su verificabilidad.
8. **Seguridad:** Se define como la capacidad de protección de la información y los datos de manera que personas o sistemas no autorizados no puedan leerlos o modificarlos. La seguridad se subdivide en: **Autenticidad, Confidencialidad, Integridad, No repudio, y Responsabilidad.** Al ser este tema tratado en otros temas no se ampliará su significado. Simplemente apuntar que responsabilidad tiene el mismo significado que trazabilidad.

La equivalencia entre trazabilidad y responsabilidad radica en que, cualquier cambio del estado de un sistema, por ejemplo, en sus datos, ha de poder ser trazado, tanto el hecho del cambio, como el usuario o persona que ha realizado el cambio. Gracias a esto, se puede definir quién ha realizado el cambio en el sistema y se pueden determinar posibles responsabilidades del mismo.

1.2 Motivos para centrarse en la calidad en el software

Que la calidad del software sea un factor decisivo es difícil de ver, sin embargo, la falta de calidad como factor decisivo para el fracaso es algo bastante fácil de observar.

Existen muchos ejemplos de problemas en diversos desarrollos, que por falta de calidad fueron experimentos fallidos.

Por ejemplo, la sonda Phobos 1 tuvo un lanzamiento correcto y durante dos meses tuvo un funcionamiento perfecto hasta que dejó de emitir señal. Tras analizar la fuente del problema, un problema en un signo en una operación (un + por un -) le dejó sin posibilidad de orientar los paneles solares hacia el Sol, lo que la dejó sin energía.

Tiempo más tarde, se averiguó que el problema vino por una excepción al proceso de calidad por lo apretado de los plazos.

Además, diversos estudios, como el presentado en su libro "Software estimation: demystifying the Black Art", Steve McConnell define que, en función de la etapa de desarrollo, el coste de arreglar un fallo en un software va escalando en un factor de 5. De tal manera, que, si pasamos de la fase de definición de requisitos, a la construcción, un fallo puede suponer un coste de 625 unidades monetarias, en comparación con un arreglo en la definición de especificaciones.

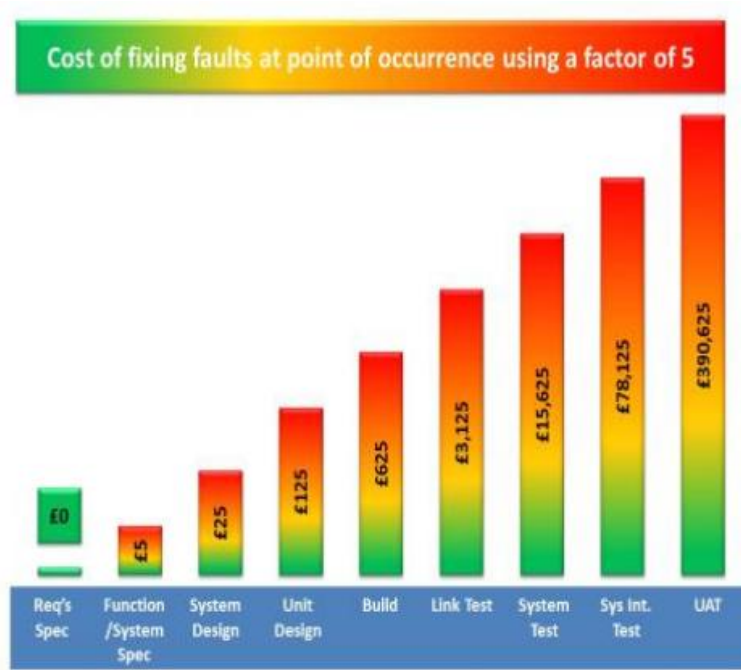


Ilustración 1. Coste de arreglar fallos según el momento de ocurrencia en el desarrollo.

De todo lo anterior se puede concluir que la calidad del software es algo importante. Y como cosa importante ha de ser incorporado en los procesos de desarrollo software de la organización, siguiendo criterios de racionalidad económica.

1.3 Aseguramiento de calidad vs control de calidad

Lo primero que tenemos que saber, es que ambas funcionalidades están dentro de una Gestión de Calidad.

La gestión de calidad son las actividades coordinadas para establecer la política de calidad, los objetivos de la calidad, la planificación, el control, el aseguramiento y la mejora de calidad.

Una vez puesto en contexto, pasamos a definir aseguramiento y control.

- Según la norma ISO 9000:2005 la definición sería la siguiente:

Aseguramiento de calidad: Parte de la gestión de la calidad orientada a proporcionar confianza en que se cumplirán los requisitos de calidad. Planes de desarrollo de procesos, auditorías de sistemas...

Control de calidad: Parte de la gestión de la calidad orientada al cumplimiento de los requisitos de la calidad. Revisión de entregables, testing...

- Si lo queremos definir en términos más cotidianos podríamos decir:

Aseguramiento de calidad: Son las acciones planificadas necesarias para proporcionar la confianza de que la fabricación, desarrollo, modificación o reparación de un producto o servicio se ha completado de manera eficiente y acorde con lo especificado.

Control de calidad: Actividades utilizadas para comprobar que el producto está diseñado, producido e instalado acuerdo con los estándares y con el nivel de calidad requerido por el diseño

Adentrándonos más en cada uno de estos aspectos podemos decir:

El aseguramiento de la calidad (QA) tiene como objetivo prevenir los defectos con enfoque en el proceso de realización de productos. Otro objetivo sería mejorar los procesos de desarrollo y de pruebas para evitar errores o defectos. El aseguramiento es responsabilidad de todo el equipo involucrado en el desarrollo del producto, en el caso de un programa SW, sería el equipo que programa el SW.

El control de calidad (QC) tiene como objetivo identificar los defectos tanto durante como después de la elaboración y antes de la entrega final al cliente, y ayudar a corregirlos. Este sería un proceso reactivo. El control de calidad es responsabilidad de un equipo específico. Este equipo tiene que ser diferente a al equipo de desarrollo para que sea completamente objetivo a la hora de sacar defectos. En el caso de un programa SW, sería un equipo QA que certifica la calidad del programa.

A modo de resumen se podría decir que el aseguramiento está orientado en la prevención de defectos y el control en la detección de los mismos.

Ambas funcionalidades son complementarias para que el producto final sea un producto de calidad para el cliente.

1.4 Estándares de testing.

Existen muchos marcos y estándares de calidad para ingeniería del software. Muchos de ellos se centran en el proceso de desarrollo del software, como un conjunto de pasos que hay que dar para llegar de una manera apropiada a un producto software. Sin embargo, nosotros nos vamos a centrar en aquellos estándares que hablan en concreto sobre el producto, es decir, la calidad del software final.

Hoy en día, existe una mezcla de estándares “de iure” y otros “de facto”. Históricamente, los estándares “de facto” han podido adaptarse más rápido y se han extendido mucho más deprisa, a pesar de los esfuerzos de las organizaciones de estandarización de crear normas que puedan ser usadas y certificadas en las empresas que desarrollan software.

1.4.1 Estándares de testing de “iure”

ISO/IEC 25000: La serie ISO/IEC 25000 también llamada SquaRE (System and Software Quality Requirements and Evaluation) es una ISO orientada al proceso, no al producto, que sustituye a las series ISO/IEC 9126 (Software Product Quality) e ISO/IEC 14958 (Software Product Evaluation).

Se divide en varias series, a saber:

- ISO/IEC 2500n, Gestión de la calidad.
- ISO/IEC 2501n, Modelo de calidad.
- ISO/IEC 2502n, Métricas de calidad.
- ISO/IEC 2503n, Requisitos de calidad.
- ISO/IEC 2504n, Evaluación de la calidad.

Dentro de esta ISO, la calidad se la ve como una amalgama entre los requisitos y los procesos de calidad en el desarrollo. Mediante la medición de forma objetiva los procesos y los requisitos, se intenta dar un indicador de calidad del mismo. Este enfoque no tiene en cuenta las pruebas y formas de trabajar de la industria, cada día más orientada a metodologías ágiles. La última publicada es la ISO/IEC 25010:2011.

ISO/IEC/IEEE 29119: ha sido el primer intento por parte de las organizaciones de estandarización de aunar los criterios de “facto” de la industria. Está desarrollado por el grupo ISO/IEC JTC1/SC7 Loring

Group 26. Se basa en los procesos de ISO/IEC 33063: Process Assessment Model, además sustituye a los siguientes estándares:

- IEEE 829 Documentación de Pruebas.
- IEEE 1008 Test unitarios.
- BS 7925-1 Glosario de términos de pruebas software.
- BS 7925-2 Norma de pruebas de componentes software.

Este estándar está dividido en las siguientes partes:

- ISO/IEC/IEEE 29119-1 Conceptos y definiciones.
- ISO/IEC/IEEE 29119-2 Proceso de pruebas.
- ISO/IEC/IEEE 29119-3: Documentación de Pruebas.
- ISO/IEC/IEEE 29119-4: Técnicas de pruebas (en la etapa de diseño).
- ISO/IEC/IEEE 29119-5: Testing basado en palabras clave.

A pesar de los esfuerzos realizados por las organizaciones de estandarización, desde la comunidad de testing agrupada en el Context Driven Testing no se ha aceptado puesto que, según ellos dicen, no se han incorporado su punto de vista. Esto es así ya que el Context Driven Testing establece que no existe método o proceso perfecto para realizar un proceso de pruebas, con lo cual podrían estar directamente en contra de la estandarización.

En la referencia bibliográfica 14 se puede ampliar más sobre la Context Driven Testing. En la referencia bibliográfica 15 se puede la respuesta de la asociación ISO.

Además, desde los estándares dominantes actuales, como el ISTQB y su modelo de certificación de madurez de las empresas, el TMMI, se critica que esta ISO solo llegaría cubrir todos los aspectos del nivel 1 y gran parte del nivel 2. Es importante destacar que el modelo TMMI tiene 5 niveles, en equivalencia a los niveles CMMI.

1.4.2 Estándares de testing de “facto”

Dentro de los estándares aceptados por la industria, tenemos los de la asociación ISTQB y los propuestos por la empresa Sogeti. La característica principal en la que destacan los siguientes estándares es que se puede certificar a personas en ellos, lo que permitiría poder exigirse en perfiles a la hora de contratar.

TMAP Next: Testing management Process de la empresa Sogeti. Basado en BDTM (Business Driven Test Management). Si se quiere ampliar información, en la referencia bibliográfica 5 se puede ampliar.

ISTQB: International Software Testing Qualifications Board. Es una asociación sin ánimo de lucro afincada en Bruselas desde el año 2002, cuyo objetivo fundacional ha sido definir y mantener un “cuerpo de conocimiento” que permita a los probadores ser certificados en el conocimiento de las

mejores prácticas, conectando la comunidad de pruebas software internacional y fomentado la investigación. Es el estándar de “facto” más usado en la industria

Excepto para la parte de testing ágil y de herramientas, se seguirán las guías del ISTQB, que se pueden ampliar con el libro indicado en la referencia 4 de la bibliografía.

1.4.3 Principios de las pruebas software

Independientemente de la aproximación, existen una serie de reglas o principios de las pruebas software que nos servirán para entender mejor cómo aproximarnos a ellas de una manera adecuada.

1. Es imposible demostrar la ausencia de defecto: las pruebas están orientadas siempre a buscar defectos en el software que reduzcan los costes de arreglarlos en etapas posteriores de las pruebas.
2. La ausencia de fallos no prueba que el software sea correcto: pueden existir problemas en el procedimiento o el diseño de las mismas que hagan que no se puedan descubrir los errores.
3. Las pruebas exhaustivas no son posibles. Analizar todo y cada uno de los casos posibles, además de ser muy costoso, se puede volver imposible, ya que el número de pruebas a realizar sería demasiado alto y el coste de hacer pruebas podría sobrepasar al coste de encontrar los errores, que es el elemento identificativo de las pruebas. Se ha de buscar una estrategia donde se encuentre un compromiso entre una explosión combinatoria de pruebas y no realizar pruebas.
4. Se han de realizar pruebas tempranas: Tal y como se ha comentado anteriormente, cuánto antes se descubra el fallo, el costo de arreglarlo se reduce. Cuanto más se avance en el proceso de desarrollo, el coste aumenta de manera exponencial con un exponente de 5.
5. Principio de localización en las pruebas: Este principio establece que, si en un módulo se ha encontrado un defecto, es muy probable que existan muchos más que seguramente no han aparecido, o que, si han aparecido, aparezcan todos juntos en el mismo sitio. Para siguientes ciclos de pruebas, se debería rediseñar el plan de pruebas analizando más en profundidad donde se han encontrado errores.
6. La paradoja del pesticida: Si hacemos varios ciclos de pruebas sin cambiar las mismas, llegará un momento que dejarán de ser útiles para conseguir su propósito, que es encontrar nuevos fallos en el software. Eso no significa que deban dejarse de hacerse esas pruebas, sino que hay que diseñar nuevas pruebas cada cierto tiempo para evitar encontrarse nuevos problemas.
7. Las pruebas dependen del contexto: En función del entorno (Desarrollo, integración, producción...) las pruebas no han de ser iguales, ni se han de ver en las mismas circunstancias. Cuando una herramienta está en producción, solo tendría que pasar una serie de pruebas básicas para verificar que el despliegue ha sido el adecuado. Sin embargo, en Preproducción se puede realizar pruebas de carga y estrés que en producción no tendría sentido, pues afectarían al funcionamiento normal de la aplicación.
8. La calidad no es solo ausencia de errores: Como punto final, es importante hacer ver que, una vez asegurada la ausencia de errores, no podemos afirmar que el software esté libre de fallos, puesto que con ello no nos hemos asegurado de que lo desarrollado sea lo que el usuario desea, siendo la adecuación de la funcionalidad a las necesidades del usuario un parámetro de calidad básico.

1.5 Conclusiones

Intuitivamente, la calidad se puede definir como la suma de la ausencia de fallos, la conformidad con los requerimientos y la satisfacción del usuario.

Históricamente, a la calidad se le ha declarado como un conjunto de atributos, que son: adecuación funcional, eficiencia, compatibilidad, usabilidad, fiabilidad, seguridad, mantenibilidad y portabilidad.

El estándar ISTQB es el estándar “de facto” más extendido y utilizado. Es un estándar de control de calidad, no de aseguramiento de calidad.

A la hora de realizar las pruebas, es importante saber que existen una serie de principios que aplican directamente y que se han de tener en cuenta a la hora de realizar los ciclos de pruebas.

2 Planificación, Estrategia y gestión de las pruebas

En este apartado se van a analizar los elementos más importantes para realizar una planificación de las pruebas, definir su estrategia, al igual que tener un breve conocimiento del ciclo de vida de las pruebas, dentro del ciclo de vida del software.

2.1 Ciclo de vida genérico

En general, los procesos de pruebas software, como elemento para asegurar la calidad, tienen un ciclo de vida como el que se muestra en la siguiente imagen:

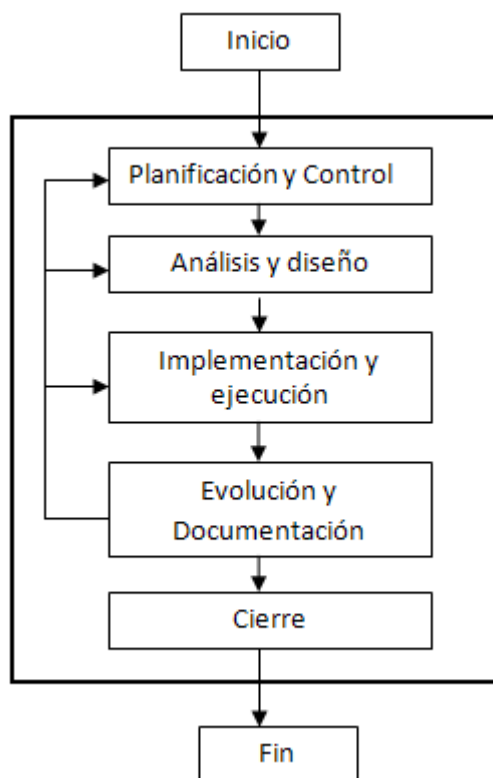


Ilustración 2 ciclo de vida de pruebas genérico

Planificación y el control: En esta fase se definen cuántas personas se van a dedicar a comprobar la calidad del sistema. Además, se definirá desde qué momento en el ciclo de desarrollo deberán empezar a realizar su trabajo, al igual que definir sobre qué elementos se van a establecer los métodos de calidad.

También en esta fase se deben determinar, instalar y configurar los entornos en los que se realizarán las pruebas, al igual que se establecerán medidas de monitorización y control, tanto del trabajo de las personas como del uso que se les están dando a las máquinas del entorno que usará calidad. A este entorno se le suele llamar Calidad o Certificación y es el paso previo para pasar al entorno de Preproducción, **aunque el nombre varía según los entornos disponibles en cada organización y su plan de calidad.**

Análisis y diseño: En esta fase se realizan, en función del número de pantallas, de los campos, y de las funciones, una estimación del número de pruebas a crear y sus casos de pruebas.

Una vez realizada la estimación se empezarán a diseñar las pruebas, dejando escrito paso a paso lo que hay que realizar, ya que muchas veces la persona que diseña las pruebas no tiene porqué ser,

necesariamente, la misma persona que las ejecuta. Luego se han de dejar las instrucciones de una manera clara y precisa.

Este paso de diseño es probablemente el más importante, puesto que se considera que una prueba pasada, es una prueba que implica un funcionamiento correcto del software y que ese funcionamiento es el que espera el usuario. Los criterios de aceptación pueden y deben ser definidos en pruebas software, más o menos exhaustivas.

Implementación y ejecución: Una vez diseñadas e implementada, se empezarán a realizar un primer ciclo de pruebas, donde se verán los fallos encontrados, utilizando las técnicas que se definirán más adelante, para que una vez detectados, se puedan comunicar a los desarrolladores para que las corrijan.

Evolución y documentación: por cada ciclo de pruebas, habrá que crear una documentación donde se resumen los resultados de las pruebas y se realizan pequeñas estadísticas con posibles mejoras o puntos donde puede haber errores.

Este ciclo se puede repetir la cantidad de veces necesaria, hasta que se considere que el software está listo para promocionarse y, una vez promocionado, para mejorar la eficacia de las pruebas, evitando ejecutar pruebas que ya no tengan validez real, ya sea porque los requisitos han cambiado o porque probar cosas estables que no se han tocado, es una forma de desperdiciar tiempo y recursos.

2.2 Gestión de las pruebas

2.2.1 Roles

Para comenzar, vamos a definir los roles y el papel juega cada uno.

Hay dos perfiles básicos: Responsable de pruebas (Test leader) y Probador (Tester):

1. Funciones del responsable de pruebas:

- 1.1. Coordinar la estrategia y el plan de pruebas.
- 1.2. Escribir o revisar una estrategia.
- 1.3. Planificar las pruebas.
- 1.4. Iniciar la especificación, preparación, implementación y ejecución de las pruebas, monitorizar los resultados y comprobar los criterios de finalización.
- 1.5. Adaptar la planificación en base a los resultados y tomar cualquier acción necesaria.
- 1.6. Gestión de configuración de SW.
- 1.7. Introducir métricas para la medición del progreso de pruebas y evaluar la calidad de las pruebas y del producto.
- 1.8. Decidir que debería automatizarse.
- 1.9. Seleccionar herramientas y organizar la formación.
- 1.10. Decidir sobre el levantamiento del entorno de pruebas.
- 1.11. Escribir informes resumen de las pruebas.

2. Probador

- 2.1. Revisar y contribuir en los planes de prueba.
- 2.2. Analizar, revisar y evaluar los requisitos de usuario para valorar que puede ser probados.
- 2.3. Crear especificaciones de pruebas.
- 2.4. Montar el entorno de prueba.
- 2.5. Preparar y adquirir datos de prueba.

- 2.6. Implementar pruebas a todos los niveles, ejecutar y registrar y evaluar los resultados y documentar.
- 2.7. Usar herramientas de administración de pruebas y probar.
- 2.8. Automatizar pruebas.
- 2.9. Medir las prestaciones de componentes del sistema.
- 2.10. Revisar las pruebas desarrolladas por otros.

En función del grado de especialización, los roles se pueden dividir a su vez en: Analista, Diseñador, Automatizador, Administrador de sistemas de pruebas, Ejecutor de pruebas. A continuación, se van a describir los principales roles y tareas de cada perfil:

3. Analista y Diseñador

- 3.1. Analiza, revisa y evalúa los requisitos de usuario, las especificaciones y los modelos.
- 3.2. Propone las pruebas necesarias y establece el orden de su ejecución.
- 3.3. Conocimientos: Desarrollo de pruebas, Ingeniería del SW y métodos de análisis y especificación.

4. Automatizador

- 4.1. Compruebas las posibilidades de automatización y las lleva a cabo.
- 4.2. Conocimientos: Como probador, diseño de pruebas y automatización, programación y herramientas.

5. Administrador de sistemas (perfil que habitualmente está en los equipos de sistemas)

- 5.1. Implanta el entorno de pruebas y lo gestiona.
- 5.2. Conocimientos: Administración de Sistemas, BBDD y Redes, instalación y gestión del sistema.

6. Ejecutor de pruebas

- 6.1. Ejecuta las pruebas en función de las especificaciones.
- 6.2. Conocimientos: Generales de IT y básicos de pruebas, manejo de herramientas, experiencia en ejecución de pruebas y conocimientos acerca del objeto a probar.

2.2.2 Planificación

La planificación se realiza en un documento llamado plan de proyecto de pruebas o plan maestro de pruebas. Es en ese documento donde se recoge la estrategia y la posible planificación a nivel global para cada nivel de pruebas (regresión, unitarias...). La norma IEEE 829 "Standard for test Documentation" recomienda que el plan maestro de pruebas tenga la estructura siguiente:

- 1. Identificación del Plan.
- 2. Introducción.
- 3. Objetos de prueba.
- 4 Características a probar.
- 5. Características sin probar.
- 6. Estrategia.
- 7. Criterios de aceptación y rechazo.
- 8. Criterios de interrupción y reinicio.
- 9. Entregables.
- 10. Tareas.

11. Necesidades de entorno.
12. Responsabilidades.
13. Personas y formación.
14. Riesgos.
15. Aprobación y autorización.

Si se quiere bajar al detalle de cada tipo de pruebas, se utilizarán planes de pruebas por separado.

El plan de pruebas es el documento donde se presenta el alcance, enfoque, recursos y planificación de las actividades de prueba. Se identifica los elementos y características a probar, las tareas, el personal encargado de cada tarea y los riesgos asociados.

Estos documentos están vivos. Conforme se va ganando en conocimiento y van cambiando las necesidades de proyecto los documentos van cambiando.

Entre las actividades que se pueden planificar, podemos destacar las siguientes:

- El alcance de las pruebas y los riesgos.
- Establecer las estrategias de pruebas.
- Integrar y coordinar las actividades dentro del ciclo de vida software.
- Establecer recursos, roles y planificar cuándo han de entrar en acción cada persona, al igual que las condiciones para empezar a probar y dejar de probar.
- Seleccionar métricas de monitorización y avance.

Para poder realizar los apartados que define el plan maestro, es necesario realizar una estimación del tiempo, con ello se podrá definir una estrategia de pruebas, que nos permitirá buscar los recursos adecuados y, junto con una gestión de los riesgos, estaremos en disposición de realizar las pruebas.

Con todo esto ya podríamos realizar una planificación exitosa, puesto que ya hemos podido definir la estrategia de pruebas, conocemos los riesgos del producto, tenemos distintas técnicas para analizar donde focalizar el esfuerzo y con todo esto podemos realizar la estimación en esfuerzo, habitualmente en horas/persona.

2.2.3 Estimación de las pruebas.

Para realizar una buena planificación es indispensable realizar una estimación del tiempo necesario para realizar las pruebas y la documentación asociada.

Para estimar existen dos aproximaciones:

- **La aproximación basada en métricas:** basadas en estimar el esfuerzo de pruebas basándose en métricas de proyectos previos o similares y reutilizando valores típicos.
- **Aproximación basada en experiencia:** en donde la estimación queda a cargo del propietario de las tareas o expertos en estimación que se contraten.

2.2.4 Estrategia de pruebas

Al ahora de definir la estrategia de pruebas, es interesante conocer los enfoques que se pueden dar en función del momento temporal en el que se comienza el diseño de las mismas:

- **Enfoques predictivos:** en los que las pruebas se diseñan tan pronto como se pueden
- **Enfoques reactivos:** en los que el diseño se realiza después de que el software haya sido diseñado.

Una vez conocemos en qué momento del proceso de desarrollo se ha comenzado el diseño de las pruebas, necesitamos saber qué otros tipos de enfoque podemos realizar para definir la estrategia de pruebas y con ello realizar los diseños de las mismas.

Entre las estrategias para el diseño de pruebas, se pueden seguir los siguientes enfoques:

- **Enfoques analíticos:** Las pruebas se orientan a las áreas de mayor riesgo. Por ejemplo, bajo este enfoque, en un banco, el grueso de las pruebas sería en lo referente al movimiento de dinero, transferencias e ingresos.
- **Enfoques basados en modelos:** Se utiliza información estadística acerca de las ratios de fallo o uso. Siguiendo con el mismo ejemplo anterior, el banco sacaría estadísticas para ver que partes de su aplicación son las que más fallan, por ejemplo, la consulta de saldo. Con esa información se diseñaría un plan de pruebas donde, aparte de comprobar el funcionamiento general de la aplicación, se pondría foco en diseñar pruebas exhaustivas en la parte de consulta de saldo.
- **Enfoques metódicos como los basados en fallos:** Basados en la experiencia, listas de comprobación... En este caso se basa en la experiencia del Tester. Por ejemplo, si el tester sabe que el grueso de incidencias está en la parte de transferencias, al diseñar el plan de pruebas hará foco en esta parte. Este enfoque es totalmente subjetivo.
- **Enfoques que cumplen procesos o estándares:** por ejemplo, cumplir con las normas de aviación en cada caso.
- **Enfoques dinámicos y heurísticos:** donde se adaptan al proceso de desarrollo y, tras un análisis, se añaden o se cambian nuevas pruebas. Esta técnica se aplica en la metodología agile. En este caso se empieza diseñando sobre lo que se conoce de la aplicación, en este momento poco, y se ejecutan las pruebas. Con las conclusiones de estas primeras pruebas se mejoran y/o se añaden pruebas y se vuelven a ejecutar. Y así hasta que el proyecto finalice.
- **Enfoques de consulta:** La cobertura de pruebas está dirigida por el asesoramiento y guía de expertos. En este caso, habrá dentro del equipo de calidad uno o varios expertos los cuales dirigen y revisan las pruebas. Estas personas tienen un gran conocimiento de la aplicación, por lo que son los que dicen que se tiene que probar y cómo.
- **Enfoques centrados en regresión:** Incluyen la reutilización de material de pruebas existentes. La mayoría de las compañías tiene un plan de regresión que se actualiza con los nuevos desarrollos de la aplicación y que se realiza siempre para certificar la aplicación antes de pasar a producción. En este caso hay poco diseño de pruebas y mucha reutilización de pruebas ya existentes.

Como se puede entender, en la definición de la estrategia se pueden utilizar varios enfoques, según se considere necesario.

2.2.5 Riesgos

El **riesgo** es la posibilidad de que ocurra un evento con consecuencias no deseadas, un problema potencial. El nivel de riesgo vendrá por la probabilidad de que ocurra un evento y por el impacto.

Los riesgos típicos en un proyecto viene por:

1. Factores organizativos:
 - 1.1. Falta de recursos.
 - 1.2. Problemas del personal.
 - 1.3. Estimaciones desde dirección no realistas.
2. Factores técnicos:
 - 2.1. Requisitos incorrectos.

- 2.2. Baja calidad del diseño o del código.
- 2.3. No disponibilidad del entorno de pruebas.
- 3. Riesgos asociados al proveedor:
 - 3.1. Posibles problemas contractuales.
 - 3.2. Mala calidad de los recursos asociados.

Hay un tipo especial de riesgo, que es el riesgo asociado al producto, que se pueden definir como aquellas áreas de fallo potencial del SW. A la hora de priorizar los esfuerzos en pruebas, es absolutamente necesario analizar qué partes del software son más críticas para su funcionamiento y con ello ayudar en la definición de la estrategia y por ende, de la planificación.

Para gestionar este tipo de riesgos, es necesario identificar, analizar y priorizar los riesgos, desde la fase de planificación. Una vez detectados, será necesario evaluar los niveles de profundidad en las pruebas, el orden de ejecución y los métodos de pruebas adecuados al riesgo que se quiere mitigar.

2.3 Ciclo de vida de ejecución de las pruebas.

Una vez realizado el plan maestro de pruebas, donde se define la planificación y la estrategia de pruebas, y una vez diseñado el plan de pruebas para cada tipo de pruebas definidas siguiendo la estrategia de pruebas, es la hora de preparar los datos para la ejecución.

Podríamos definir como datos de prueba todas aquellas entradas con las que es alimentado el sistema para ser probado en períodos de evaluación.

La mala gestión de los datos de prueba puede desembocar en:

- Detener o retrasar la ejecución de pruebas por la mala claridad de los datos
- Cobertura incompleta en el proyecto de pruebas.
- Reportar defectos inválidos a causa de una incorrecta configuración/preparación de datos.
- Dejar de detectar defectos.
- Manejo inadecuado de datos de producción que pudieran propiciar problemas legales (privacidad, seguridad).

La preparación de los datos siempre es una de las tareas más complicadas. Normalmente dentro del equipo de calidad existe una persona encargada de este proceso. Una vez que tengamos los datos de prueba, podemos pasar a la ejecución de las pruebas.

En la ejecución, se recoge cada documento de diseño de pruebas y se ejecutan las pruebas por los probadores o ejecutores. Una vez que estos han dado un OK o un KO a la prueba y generan el informe correspondiente el gestor de pruebas se encarga de evaluar los resultados, para generar un informe que irá a los autores del objeto de prueba para que puedan solucionarlo.

Una vez realizada la primera ejecución de las pruebas, se inicia un ciclo de gestión de incidencias. El procedimiento de gestión de incidencias tiene que estar definido al inicio del proyecto, donde al menos se han de dejar constancia:

- Formas de notificación común a todos los interesados.
- Normas para la clasificación de las incidencias.
- Definición del proceso detallado, donde se ha de incorporar desde el inicio de la apertura de las incidencias, hacia quién se escala, dónde se ofrece una propuesta a la solución y finalizando por la aceptación de la resolución.

El tratamiento de la incidencia se realiza en función de la prioridad asignada, siguiendo el principio de que, a mayor gravedad, mayor prioridad.

Es importante destacar que el probador es el único que puede definir un error como solucionado. Sobre todo teniendo en cuenta que, si realiza bien sus tareas, una vez detectado un error, habría que cambiar las pruebas a realizar para que, aplicando el principio de localidad en las pruebas, se puedan ver otras fuentes de problemas posibles.

La decisión de si una incidencia debe ser rechazada o trabajada, debe tomarla el gestor de pruebas

2.4 Conclusiones.

Antes de empezar a probar hace falta realizar una planificación y con ella la asignación de roles, recursos y tareas, donde se establezcan principalmente: un análisis de riesgos, definir una estrategia de pruebas, una estimación de tiempos, y una asignación de tareas y personas y un procedimiento de gestión de incidencias.

La estimación podrá ser basada en métricas o en experiencia en el proyecto o de un probador experto. La gestión de riesgos, dentro del enfoque analítico, al igual que otros enfoques como los metódicos o los basados en modelos, son la base para la definición de la estrategia de pruebas, donde escogeremos donde focalizar los esfuerzos en la planificación.

Finalmente, un procedimiento de gestión de incidencias ayuda a definir y saber qué pasos se darán para resolver las incidencias y cuando se considerará una incidencia como solucionada, o cuándo será solucionada.

3 Clasificación y tipos de pruebas

La primera y gran clasificación de las pruebas está entre funcionales, es decir, aquellas que verifican cómo funciona el sistema y las no funcionales, aquellas que prueban características del sistema que no son directamente peticiones del usuario.

Así, por ejemplo, en las pruebas funcionales han de ser probados los siguientes puntos:

- Idoneidad: ¿Las funciones implementadas son adecuadas para su uso esperado?
- Precisión: ¿Las funciones presentan los resultados correctos?
- Interoperabilidad: ¿El sistema cumple con normas y reglamentos aplicables?
- Conformidad: ¿Las interacciones con el entorno del sistema presentan algún problema?
- Seguridad: ¿Están protegidos los datos/programas contra acceso no deseado o pérdida?

En las pruebas no funcionales, habría que probar, al menos:

- Pruebas de carga: Como se comporta el sistema ante unos usuarios dados
- Pruebas de rendimiento: Como de rápido es el sistema en determinadas funciones
- Pruebas de volumen / masivas: Como se comporta el sistema al procesar grandes cantidades de datos
- Pruebas de estrés: Alcanzar el máximo número de usuarios hasta la sobrecarga del sistema y comprobar si se recupera
- Pruebas de seguridad: son pruebas que permiten verificar si el sistema es robusto ante los intentos de uso mal intencionados y no previstos del mismo.

Además, en función de lo cercano al código que estén, se pueden definir las pruebas por la siguiente manera:

1. Pruebas unitarias: son aquellas que se realizan a módulos de código fuente, sin que tengan que estar referenciado necesariamente a funcionalidades finales del usuario. Se suelen utilizar en métodos o clases para que el autor del código pueda verificar su funcionalidad.
2. Pruebas de integración: son aquellas pruebas que se realizan entre distintos módulos de un sistema que han de comunicarse o relacionarse. Un ejemplo claro son los servicios web, que son desarrollados en programas distintos, pero que tienen que funcionar juntos. En las arquitecturas de microservicios son muy importantes estas pruebas, ya que los módulos se integran mediante llamadas usando protocolos de servicios Web, como Rest.
3. Pruebas de sistema: son aquellas que verifican la compatibilidad del código desarrollado con las aplicaciones del sistema donde son desplegadas: compatibilidad con el servidor de aplicaciones, con la base de datos, con las máquinas utilizadas.
4. Pruebas de aceptación del usuario: Son pruebas que se realizan con la aplicación final, y que normalmente se interacciona como lo haría el usuario. El objetivo es buscar problemas clásicos tales como problemas como casos imposibles o límite, entrada de datos no válidos, resultados de aplicaciones no esperados...
5. Pruebas de carga: son aquellas que verifican el comportamiento del sistema ante una sobrecarga de usuarios y/ o datos.
6. Pruebas de seguridad: son aquellas pruebas que se realizan para comprobar que usuarios maliciosos no pueden aprovechar usos no esperados de la aplicación para acceder a información o funcionalidades a las que no deberían tener acceso.
7. Pruebas de regresión: Conjunto de pruebas que se realizan después de una modificación que aseguran que no se han introducido defectos en el código ya existente como resultado de

cambios realizados en el código. Se realizan cuando el software o su entorno han sido cambiados.

8. **Pruebas de usabilidad:** son aquellas en las que se verifica que el interfaz de usuario es limpio, ordenado, responsivo e intuitivo. Son muy importantes sobre todo para aquellas aplicaciones orientadas al público en general, que no está dispuesto a gastar mucho tiempo y esfuerzo en aprender a utilizarlas.
9. **Pruebas de accesibilidad:** las que se realizan para verificar que personas con algún tipo de discapacidad pueden tener la posibilidad de acceder a la información. Para ello se puede utilizar estándares como los WCAG.

Como complemento a los métodos mencionados anteriormente nunca se debería prescindir de las especificaciones intuitivas de casos de prueba.

Las pruebas engloban diferentes métodos y procedimientos que pueden ser combinados con sentido.

3.1 Técnicas estáticas.

Las técnicas estáticas engloban procedimientos de análisis manual o realizado por herramientas, en los que no se ejecuta el objeto de prueba, si no que se analiza cómo se ha realizado para intentar mejorar la calidad del software, entiendo el mismo como el conjunto de código más documentación.

El objetivo de las pruebas estáticas es encontrar errores/incidencias mediante un examen manual del código o de cualquier otra documentación del proyecto sin ejecutar. Estas pruebas se pueden realizar en todas las fases del ciclo de vida del software.

Gracias a estas técnicas, los defectos se van detectando en fases tempranas y por lo tanto la corrección es menos costosa.

A continuación, vamos a estudiar las técnicas estáticas más importantes.

3.1.1 Revisiones

La revisión se puede definir como el análisis de elementos escritos, que no generan funcionalidad final y cuyo objetivo es revisar la adecuación de los mismos a los criterios de calidad descritos. No se espera un resultado de OK o KO, sino analizar cómo se están realizando las implementaciones y/o la documentación.

Así, por ejemplo, tenemos revisiones de requisitos, de diseño o inspecciones de código.

Entre los defectos típicos más fáciles de encontrar con esta técnica son: desviaciones de estándares, defectos en los requisitos o en el diseño del código, mantenibilidad insuficiente y especificaciones del interfaz incorrectas.

3.1.1.1 Roles en las revisiones

Las revisiones se suelen realizar en una o varias reuniones y participan los siguientes roles, que no personas, ya que una persona puede adoptar varios roles, son los siguientes:

- **Gestor:** Encargado de organizar las revisiones, asignar recursos y distribuir tareas.
- **Moderador:** Es neutral, dirige la revisión y modera entre los diferentes puntos de vista.
- **Autor:** Elabora el objeto de prueba o elemento a ser probado. Debe asumir las modificaciones al mismo debido a la evolución del ciclo de vida del producto.

- **Supervisor:** Reciben con anterioridad el objeto de prueba y verifican su calidad.
- **Documentador:** Prepara una lista de todas las consideraciones realizadas en la revisión. Se podría considerar que hace de secretario, al levantar acta.

3.1.1.2 Fases de la revisión

El proceso de revisión elementas se compone de las siguientes fases:

Planificación: en esta fase participan el gestor y el moderador.

En esta fase se realiza:

- La organización de las revisiones.
- La planificación de los plazos (moderador) y de los recursos(gestor).
- La elección de las personas participantes y asignación de roles (gestor).
- Selección de los objetos de prueba.
- La definición de los criterios de entrada y salida para las revisiones más formales. Se entiende por criterios de entrada lo que hacen que se empiece a probar y los de salida los que hacen que se pare de probar.

Preparación organizativa: Aquí participa sólo el moderador.

Existe una reunión previa (o reunión de kick-off) donde se distribuyen los objetos de prueba, se distribuye toda la información relevante para la evolución y se comprueban los criterios de prueba y se reparten tareas. La comprobación de los criterios de entrada se realizaría en el caso de revisiones más formales.

Preparación individual: Participa el supervisor.

Se refiere a la preparación de los participantes para la reunión, realiza la revisión de la documentación y/o objeto de prueba, anotando los posibles defectos, dudas y comentarios.

Reunión de revisión:

Se reúnen todos los implicados para exponer los problemas que se han encontrado en el análisis de los objetos de prueba. Se ha de evitar atacar al autor de los mismos, y se ponderará la importancia o no de su corrección.

Retrabajo: Participa el autor.

Aquí es donde el autor de los objetos de pruebas, a la vista de lo expuesto y acordado en las reuniones de revisión, corrige o mejora los objetos de prueba.

Seguimiento: Participan el gestor y el moderador.

Tanto el gestor como el moderador, a la vista de lo aecido en las fases anteriores y con la documentación generada, deciden si seguir con un ciclo más de las revisiones o finalizan la misma.

3.1.1.3 Tipos de revisión

En función de lo formales que sean, tenemos los siguientes tipos de revisiones, según se define en la IEEE 1028:

1. Inspección

- 1.1. Modo de proceder **altamente formal**. El objetivo es encontrar defectos.
- 1.2. Moderador independiente, distinto al autor.

2. Walkthrough

- 2.1. También se distribuye la documentación con antelación.
- 2.2. El autor es el mismo que el moderador y es el que presenta el objeto de prueba.
- 2.3. Propósitos: aprender, ganar conocimiento y encontrar defectos.
- 2.4. Menor esfuerzo, el autor tiene mucha influencia y es el encargado de la elaboración posterior.

3. Revisión técnica

- 3.1. Comprobación de los aspectos técnicos del objeto de prueba. Sin autor.
- 3.2. Propósitos: tomar decisiones, evaluar alternativas, encontrar defectos y comprobar conformidad.
- 3.3. Eficiente con personas expertas, gran esfuerzo de preparación, búsqueda estructurada de errores.

4. Revisión informal

- 4.1. La variante más simple, a menudo iniciada por el autor.
- 4.2. Los resultados se transmiten en una lista.
- 4.3. Organización y desarrollo se realizan rápidamente, el coste es bajo, no se documentan los resultados ni el proceso.

3.1.2 Análisis estático o dictámenes apoyados por herramientas

Dentro del análisis resto de las técnicas estáticas, tenemos el análisis estático del código o también llamado “apoyado por herramienta”. El uso de estas técnicas nos permite encontrar errores en el software al igual que obtener métricas de la calidad del mismo.

3.1.2.1 Complejidad ciclomática

El flujo de control es el análisis de todas las posibles ramas de ejecución, representadas por grupos “*if-then-else*” y por bucles.

El objetivo de los flujos de control es múltiple: se puede utilizar para encontrar ramas muertas, bucles infinitos. Un uso muy importante es el cálculo del número ciclomático.

El número ciclomático nos define la complejidad del software. Se suele calcular las de los siguientes elementos: método, fichero, clase y la total. Dentro de un método, suele considerarse complejo si el resultado es mayor a 10.

La forma de calcular la complejidad ciclomática es la siguiente: cuando se ve un bucle o una sentencia “*case*”, o “*if-then-else*” sin ser la última del método, se le añade uno a un contador. Cuando se acaba el elemento a analizar, se muestra el resultado del contador y se reinicia para analizar el siguiente elemento.

3.1.2.2 Código duplicado

Una de las formas de mejorar la mantenibilidad del código es reducir el código duplicado. Existen herramientas que analizan los métodos, estableciendo un análisis de la entrada y la salida y, si ven un posible duplicado, avisan de ello para que pueda ser optimizado en alguna iteración del desarrollo.

3.1.2.3 Ciclos entre paquetes

Un ciclo entre paquetes es la situación que se da cuando se realizan llamadas a métodos, interfaces o módulos desde un paquete “Invocador” a un paquete “Receptor”, y este, para responder a la llamada, llama a un método, interfaz o módulo del paquete “Invocador” originario de la petición.

El análisis de los ciclos entre paquetes permite que se pueda ver la calidad de nuestro diseño. Un diseño con un 0 % de ciclos entre paquetes es un diseño sin acoplamiento. Si el porcentaje empieza subir mucho, podría ser necesario parar y repensar la arquitectura.

3.2 Técnicas dinámicas o técnicas de diseño de pruebas.

Las técnicas dinámicas para la creación de pruebas son aquellas que buscan que, dados unos elementos de entrada y un estado de la aplicación, el resultado de cualquier operación sea el esperado.

Para ello, es necesario una definición formal de requisitos, aceptada por el usuario, y cuyo contenido se utiliza como base para definir las pruebas. También son necesarios unos datos de entrada y un estado interno controlado de la aplicación para que se puedan realizar exitosamente.

Tanto los datos, como el estado interno de la aplicación, como las entradas al sistema y la operación han de estar documentados en el caso de prueba y su resolución ha de ser coherente con la definición formal de requisitos. El documento que sirve para dar base a todo lo expuesto anteriormente es el documento de diseño de pruebas.

El documento de diseño de pruebas, se especifican los casos de prueba al igual que las precondiciones, los valores de entrada, el resultado esperado y las postcondiciones.

El documento de diseño de pruebas puede ser utilizado como criterio de aceptación del software.

Existe una clasificación de las técnicas dinámicas en función de si se conoce o se tienen acceso al estado interno de la aplicación, o no. Esta clasificación se define entre prueba de caja negra o funcionales o pruebas de caja blanca o estructurales. En el siguiente cuadro se puede ver un resumen de sus características:

CAJA NEGRA o FUNCIONALES	CAJA BLANCA o ESTRUCTURALES
Se utilizan principalmente para verificar que los requisitos funcionales de la aplicación se cumplen. La estructura interna del programa es irrelevante.	Se utilizan para poder verificar trozos de código. Un ejemplo serían las pruebas unitarias. Se tiene acceso al código.
Se suelen utilizar para realizar pruebas de aceptación.	Se utilizan para partes del código críticas o para auditorías externas, por ejemplo, de seguridad.
Los casos de prueba se deducen de las especificaciones	Los casos de prueba se deducen de la estructura del programa
Se utilizan en fases más tardías del desarrollo	Se utilizan en fases más iniciales del desarrollo

3.2.1 Pruebas de caja blanca

En programación, esta técnica se refiere a un tipo de pruebas de software que se realiza sobre las funciones internas de un módulo.

Estas pruebas son las que se llevan a cabo en primer lugar sobre módulos concretos, ya que con ellas se detectan errores en el código, para luego poder hacer pruebas de caja negra sobre subsistemas.

Si bien se necesita un gran conocimiento técnico y más tiempo de ejecución, esto se ve recompensado con la detección temprana de defectos que evitan su propagación en la fase de pruebas y por lo tanto ahorro de dinero y tiempo.

A continuación, se va a profundizar en las siguientes técnicas de caja blanca: cobertura de sentencias, cobertura de ramas, cobertura de condiciones y cobertura de rutas.

3.2.1.1 Cobertura de sentencias

El objetivo de la cobertura de sentencias es averiguar cuántos casos de test son necesarios para llegar a la totalidad o a una cuota de líneas de código del programa.

Para ello, se ejecutarán una serie de casos de pruebas y se verificará por qué líneas de código pasa. Se considera que por cada conjunto de instrucciones que se ejecutan siempre seguidas, se crea un nodo, el resto de saltos de ejecución en función de algún parámetro son las aristas.

La fórmula que nos da el porcentaje de cobertura es el de la siguiente imagen:

$$C_0 = \frac{\text{Nº sentencia cubiertas por caso} \\ \text{(nodos del camino)}}{\text{Nº total de sentencia} \\ \text{(nodos totales)}} \times 100\%$$

Ilustración 3 Fórmula de cobertura de sentencias

3.2.1.2 Cobertura de ramas

El objetivo es el mismo que en la cobertura de sentencias: verificar para un conjunto de casos de prueba dados, qué porcentaje de todas las ramas se atraviesa y cuales se dejan fuera. Una rama se define como cada una de las posibles alternativas de flujo del programa que se pueden dar debido a bucles o sentencias condicionales.

De tal manera, si un programa tiene 4 funcionamientos distintos en función del usuario, cada usuario será una rama del programa. Si en las pruebas solo probamos con dos usuarios, tendríamos una cobertura del 50% de las ramas del programa. En la realidad no es exactamente así, ya que existe código que está compartido para varios o todos los usuarios, pero como ejemplo ilustra adecuadamente.

El concepto de nodo y arista es el mismo que en cobertura de sentencias. La fórmula se calcula como en la siguiente imagen:

$$C_1 = \frac{\text{Nº ramas cubiertas por caso} \\ \text{(aristas del camino)}}{\text{Nº total de ramas} \\ \text{(aristas totales)}} \times 100\%$$

Ilustración 4 Fórmula para la cobertura de ramas

3.2.1.3 Cobertura de condiciones

El objetivo de este tipo de pruebas es localizar errores que se encuentren en las sentencias condicionales del código. Entendemos por sentencias condicionales instrucciones tales como “*If-Then-else*” o *Case*.

De tal manera, dado un segmento de código, se verifica que las condiciones de sentencias condicionales están cubiertos todos los casos y qué ocurre si los valores a verificar toman una serie de valores u otros.

En función de la cobertura podría ser simple (solo buscamos dos casos, en los que la condición sea cierta y en el que sea falso), múltiple (se intentan ver todos los casos por los que la estructura condicional pueda ser considerada cierta o falsa) o múltiple mínima (se observa el conjunto mínimo equivalente de casos múltiples que analicen todos los casos lógicos, reduciendo la explosión combinatoria).

La diferencia entre la cobertura de condiciones y la de ramas es la siguiente. La cobertura de condiciones está limitada a una porción del código y la de ramas está referida a todo el código, o al menos a segmentos más grandes del mismo.

3.2.1.4 Cobertura de rutas

La cobertura tiene como objetivo ver qué recorrido llevan un conjunto de datos. La forma de analizar sería la misma que en la cobertura de ramas, solo que, en vez de utilizar un conjunto simple de datos, se pueden utilizar varios y ver por donde discurren.

En este caso, la cobertura de rutas sería igual al número de rutas recorridas dividido entre el número de rutas total

3.2.2 Pruebas caja negra

Los módulos se estudian desde el punto de vista de entrada y salida, sin importar el cómo se consigue la respuesta. Están más orientadas a la funcionalidad del módulo, verificar que la respuesta que recibimos es la que deberíamos recibir.

No se requiere conocimiento técnico de la aplicación, solo requisitos funcionales de alto nivel. Son menos costosas que las anteriores y detectas errores a nivel usuario.

Tienen la limitación de que no son capaces de encontrar funcionalidad extra no especificada. Las técnicas que vamos a analizar son: las clases de equivalencia, análisis de valores límite, tabla de decisión, pruebas de estado y pruebas de casos de uso.

3.2.2.1 Clases de equivalencia

Las clases de equivalencia es una técnica basada en la reducción de la complejidad. Si fuéramos a probar una pantalla con una serie de campos en ellos, podríamos probar infinitos valores en esos campos. Para reducir la complejidad, tanto de datos de entrada como de estados posibles, intentamos buscar aquellas pruebas que nos permiten reducir la cantidad de casos de prueba, sin perder cobertura, o perdiendo la mínima posible.

Usando las clases de equivalencia, el objetivo será buscar aquel conjunto mínimo de pruebas que nos permite llegar a todos los estados posibles, valorando el caso OK y el caso KO.

Cuando se realiza el diseño de las pruebas de un programa, es importante que se analice los casos en los que se metan datos raros y salga el error esperado. Este caso es el caso KO. Una prueba que espere un KO de una aplicación, y devuelva un OK, será una prueba fallida.

3.2.2.2 Análisis de valores límite.

Habitualmente, cuando vamos a probar valores en un campo, la gran mayoría de los errores se encuentran en las fronteras de un caso a otro.

Pensemos en un programa que, si la compra es mayor o igual a 1000 euros, se hace una rebaja del 10 % al precio total, si no se tramita normalmente.

Un caso de prueba debería ser exactamente los 1000 euros, para ver si efectivamente el “if” de la sentencia está puesto como mayor estricto o como mayor igual.

3.2.2.3 Tabla de decisión

Las tablas de decisiones nos ayudan a ver las condiciones y las acciones que se esperan cuando se dan las condiciones. Es una forma de ordenar la información de manera visual para diseñar las pruebas.

	Caso de prueba			
Condiciones	A	B	C	D
¿Pago al contado?	S	S	N	N
¿Compra > 50.000€?	S	N	S	N
Acciones				
Descuento del 5 %	Aplicar	Aplicar		
Descuento del 7 %	Aplicar		Aplicar	
Importe neto de la factura	Aplicar	Aplicar	Aplicar	Aplicar

Se puede ver en la tabla como cada uno de los números de las columnas son casos de prueba,

3.2.2.4 Pruebas de estado

Los diferentes estados que pueden adoptar un objeto de prueba durante su ejecución se representan mediante un modelo de estados.

Una prueba referida al estado debería ejecutar para cada estado todas las funciones definidas en el mismo, al menos una vez.

A nivel visual se pueden establecer como tabla o como un diagrama de estados.

3.2.2.5 Pruebas de casos de uso

El objetivo es validar el sistema siguiendo los casos de uso. El caso de uso, por su definición y su forma de documentarse, puede ser utilizado de una manera excelente para ver los pasos exactos que se esperan de la aplicación, y con ello crear las pruebas necesarias.

Son muy importantes para diseñar las pruebas de aceptación de usuario, ya que los casos de uso son parte de los requisitos que tienen la aplicación.

3.3 Criterios de aceptación del software

Una vez estudiadas las técnicas, vamos a plantear el problema de la aceptación del software.

Independientemente de la aproximación, ya sea tradicional o ágil, es necesario definir cuándo vamos a dar el OK a un programa o a un código escrito. Esto tiene una importancia capital, ya que es necesario que el código llegue a producción con la mayor calidad posible.

De inicio, se podría pensar que las pruebas funcionales, su definición y aceptación por parte del equipo de pruebas y del usuario, podrían ser la única fuente de aceptación.

Esto sería equivalente a decir que, en una aplicación de gestión de correos, que se enviasen y recibiesen los correos, fuera suficiente.

Pero... ¿qué ocurriría si, efectivamente el email se enviará, pero tardara muchísimo en enviarse? ¿o que en cuánto la aplicación tuviera una carga de usuarios mayores a un umbral, se degradara el servicio de tal manera que no fuera manejable?

Es por ello, que además de utilizar las pruebas funcionales, usando técnicas dinámicas para su definición, serían necesarias pruebas no funcionales, que hemos definido que eran las siguientes: pruebas de carga, de rendimiento, de volumen, de estrés y de seguridad.

Las pruebas no funcionales nunca van a ser perfectas, pero si que son necesarias para averiguar hasta qué punto nuestro sistema es robusto ante mucha carga de trabajo, antes muchos usuarios, qué límites tiene y si tiene el rendimiento esperado para la carga de trabajo que se le supone.

Una vez definido con el usuario las pruebas funcionales y hemos verificado que la aplicación es segura, y que el rendimiento ante cargas inusuales de trabajo es satisfactorio para la carga esperada, debemos meternos en la calidad de la construcción de la aplicación.

La calidad del código está íntimamente relacionada con las técnicas estáticas. En ciertas organizaciones, se pueden definir estándares de código, es decir: Cómo se definen las variables, qué política de logs se genera, qué política de excepciones... También se pueden usar parámetros clásicos como la complejidad ciclomática, los ciclos entre módulos, códigos duplicados, porcentaje de código que está comentado...

Todas estas variables, una vez definidos unos umbrales y declarados estándares de código, nos permite tener una idea del conjunto mínimo de criterios para aceptar unos desarrollos, ya sean realizados de manera interna o realizados por una organización externa.

3.4 Conclusiones

La calidad del software se mide por la funcionalidad, y por la calidad en el diseño. Para medir la funcionalidad, tenemos las técnicas de caja negra, y para analizar la calidad en el diseño tenemos las técnicas de caja blanca siendo la más importante las revisiones.

Adicionalmente, las métricas necesarias para analizar la cobertura de las pruebas realizadas y la calidad del código generado se podrán definir mediante las pruebas de caja blanca dentro de las técnicas dinámicas.

Todas las técnicas aprendidas en este capítulo han de dar lugar a criterios que nos permitan tener de manera objetiva y cuantificable, cuándo dar el ok a un desarrollo.

4 Procesos de pruebas en metodologías ágiles

Dentro de los distintos ciclos de vida, tenemos el ciclo de vida ágil, que no está orientado tanto al proceso, si no a equipos de desarrollo.

En los procesos clásicos de ingeniería del software, primero va el desarrollo software. Una vez que el equipo de desarrollo considera que se ha terminado el desarrollo, pasaría a un equipo de calidad que evaluaría y realizaría todas las pruebas. En función del resultado de las pruebas y de la severidad de cada una, se devolvería a desarrollo para que arreglasen las funcionalidades o se permitiría promocionar a producción los desarrollos, dejándose esos errores para próximas versiones del software.

La propuesta ágil implica que, todo está integrado y que se hace en paralelo. No se termina de desarrollar todo el código y ya se está probando. Para eso hace falta una interlocución y una coordinación plena entre equipos y no basarlo en procesos reglados.

Entrando más en detalle, en lo que sería un primer sprint que podemos decir que puede ser de unas dos semanas, se deberán diseñar, documentar y automatizar aquellos test que se consideren por el equipo: unitarios, de integración, aceptación y regresión.

No será hasta que toque un sprint de subida de versión que se realizan las pruebas finales, las de seguridad y las de carga/rendimiento si así se estimase.

Es importante destacar que esta propuesta es una de las múltiples posibles, y que las metodologías ágiles dan posibilidad de adaptar todo esto a las necesidades del proyecto en concreto.

4.1 Factores de éxito para la implantación de pruebas ágiles.

Debido al cambio de perspectiva, existen una serie de puntos que a la hora de introducir el testing en las metodologías ágiles se ha de tener en cuenta, si se quiere tener una buena transición. Estos puntos, que fueron identificados, entre otros por Brian Marick son:

1. Los “Testers” son parte del equipo: No son algo aislado del grupo de desarrollo, si no que pertenecen a él, ayudan al trabajo y se les tiene en consideración.
2. Mentalidad orientada al desarrollo ágil: En la perspectiva tradicional, se hace más referencia a que los desarrollos entren en parte de la “Política de Calidad”, como si a los desarrolladores se les tuviera que controlar o que los procesos fueran más importantes que los objetivos del equipo y dar valor al usuario.
3. Automatizar las pruebas: Hay que ser muy cuidados con qué se automatiza y qué no, teniendo una mentalidad práctica. Además, el objetivo ha de ser obtener feedback rápido para ayudar a reducir a lo mínimo posible la deuda técnica.
4. Tener una visión amplia: El tester ágil ha de evitar ser un Talibán de la calidad, y saber acompañarse en un equipo donde el objetivo es lo prioritario. En caso de duda se pueden usar las matrices de testing para saber qué automatizar y qué no.

4.2 El papel del responsable de pruebas en equipos ágiles.

El proceso de desarrollo software es extremadamente complejo y difícilmente compartimentable sin pérdida de la eficacia. Las últimas metodologías como DevOps, basada en la colaboración estrecha de los desarrolladores y de la gente de operaciones (sistemas, y comunicaciones), implican necesariamente la gestión por equipos, cuya principal función es que todos los equipos y todas las personas de esos equipos han de ser multidisciplinares.

La multidisciplinariedad de cada uno de los miembros de esos equipos, implicaría en un equipo ideal que todos supieran desarrollo, todos supieran de pruebas, BBDD, servidores de aplicaciones, etc... La realidad es que los miembros de los equipos, que todos van con su disciplina aprendida, han de tener conocimientos del resto de disciplinas, por si acaso hay que hacer un esfuerzo temporal en ese aspecto.

En lo que se refiere al papel del responsable de pruebas en los equipos ágiles, tendrían que tener, idealmente, las siguientes características:

1. Conocer las técnicas para la definición de pruebas, tanto desde un nivel conceptual como a nivel técnico.
2. Además, podrán dedicarse a dar soporte al proceso de obtención de requisitos, analizando tanto el proceso de entrevista, como la de la documentación generada.
3. Entendiendo implementado y actualizando la estrategia de testing.
4. Trabajar con los propietarios del producto para definir los criterios de aceptación.
5. Realizar mediciones y reportar test de cobertura a través de todas las dimensiones definidas.
6. Asegurar que se hace un uso adecuado de las herramientas de pruebas.
7. Configurar, usar y gestionar entornos de test y datos de prueba.
8. Escribir y ejecutar los test automáticos y asegurar su ejecución.
9. Informar de los defectos a los grupos de trabajo para resolverlo.
10. Ayudar a los compañeros que mejoren su forma de trabajar con las pruebas (sobre todo unitarias y de integración, que son las típicas de los equipos de desarrollo).
11. Participar activamente en las reuniones, y ofrecer mejoras en el proceso de desarrollo.
12. Colaborar activamente con los desarrolladores y los interesados en clarificar los requerimientos, especialmente en términos no funcionales.

4.3 Frameworks ágiles.

A continuación, se van a definir una serie de técnicas y frameworks que la comunidad ágil ha ido desarrollando a lo largo del tiempo.

4.3.1 TDD

Una vez entendido el papel del responsable de pruebas en un equipo ágil, ahora toca ver técnicas orientadas al uso de las pruebas y de evaluación de la calidad en equipos ágiles.

La primera de ellas, y que ya fue implementado en las metodologías XP, es TDD. TDD (test driven development) es una técnica de diseño software que se basa en los siguientes pasos:

1. Antes de codificar hay que generar un conjunto de casos de ejemplo.
2. Una vez realizados esos casos de ejemplo, se creará un test unitario o de componente automático que verifique los casos de ejemplo.

3. Después se codificará el código para responder al ejemplo de OK, implementado en los casos de test unitarios.
4. Una vez que el test unitario da el OK al código desarrollado, verificar si hay porciones de código que duplican el trabajo. Si se localizan, refactorizar mediante el uso de patrones de diseño (Más sobre patrones de diseño en el tema 89).

Las ventajas que nos ofrece TDD, según Kent Beck en su libro *Test Driven Development by example* (2003), son las siguientes:

1. Permite código reutilizable y favorece la refactorización del código.
2. Mejora la comunicación entre compañeros y ayuda a mejorar la confianza en el código realizado por un compañero.
3. Las personas que se dedican a pruebas pueden dedicarse a labores de mayor valor añadido.
4. Al diseñar primero las pruebas, se minimiza la posibilidad de sobrescribir en el código.
5. No hay que dedicar demasiado tiempo a pensar la arquitectura, ya que el propio desarrollo de las pruebas provocará que la arquitectura vaya emergiendo mediante las refactorizaciones constantes.
6. Los test, además de probar el propio código, también deberán probar el código generado por otros compañeros, en lo que son llamados los test de integración, que también deberán ser automatizados.

4.3.2 ATDD

Cuando hablábamos de TDD nos habíamos centrado en realizar automatización de pruebas para mejorar la calidad del software desarrollado, pero en ningún caso hablábamos de la necesidad de incluir en esas pruebas automatizadas, el criterio de aceptación del usuario.

ATDD es una técnica de escritura de pruebas en la cual se espera que el usuario final defina y diseñe las pruebas que den el OK a su uso. También se le llama *Story Test Driven Development*.

Uno de los grandes problemas que existen en los desarrollos software es saber si lo desarrollado era exactamente lo que pedía el usuario. Una de las posibles soluciones es pedir al usuario que especifique, junto con el desarrollador y el encargado de calidad el criterio de aceptación o, dicho de otra manera, cuando una funcionalidad se puede dar por hecha.

Para entender la relación existente entre el desarrollador, el usuario y el responsable de pruebas se utiliza la parábola de los 3 amigos. En ella, el usuario o product owner, es la persona que expresa el problema a sus amigos, donde expone todos los problemas que tiene y cómo organizarlos. Por otro lado, está el desarrollador, que escucha al amigo y le propone una solución, de las múltiples posibles que existen. El tercer amigo, la persona encargada de las pruebas, es la encargada de escuchar el problema y la solución y buscar qué problemas hay, tanto en el proceso de explicación de los problemas como en las posibles soluciones para mejorar la solución final.

4.3.3 BDD

BDD es una evolución de TDD, que fue citado por primera vez en 2006 por Dan North en su artículo "Introducing BDD". BDD nació con la idea de desarrollar TDD y responder a las dudas principales que los programadores tenían. Éstos, no sabían qué probar y qué no probar, cuánto había que probar y qué podría permitir una prueba.

A lo largo de su artículo, Dan North establece una serie de técnicas a incorporar que permitían los siguientes hitos:

1. Utilizar las pruebas como modo de documentar los criterios de aceptación y documentar las partes del código. Esto ya lo hemos visto en ATDD.

2. Orientar el punto de vista y el nombre de las pruebas a comportamiento de negocio, en vez de al código puro. De tal manera que, si se quiere verificar que las fechas introducidas en un formulario son correctas, el método que prueba ese criterio de aceptación debería llamarse, utilizando notación Java, `fechasValidasEnFormularioPersona`.

El cambio a un paradigma de pruebas orientado al negocio, en vez de al código, permitió la evolución de las historias de usuario. En un principio, las historias de usuario tienen la siguiente estructura:

Como [Rol del usuario de la aplicación]

Quiero [Acción que quiere realizar en el sistema]

Para [Beneficio desde el punto de vista de negocio]

Y los criterios de aceptación solían estar escritos en la parte de atrás de la historia.

Por ejemplo, si estuviéramos en un cajero automático, la historia de usuario podría ser:

Título: Cliente quiere sacar dinero.

Como Cliente

Quiero Sacar dinero de mi cuenta corriente

Para evitarme tener que esperar colas en el banco.

Aceptación: [Caso OK] Que me deje sacar dinero y lo descuenta de mi saldo, [Caso KO] Que no me deje sacar si no tengo fondos.

Al ser los criterios de aceptación tan laxos, muchas veces la calidad del software no era tan buena. Muchas veces no se contaba con el usuario para definirlos adecuadamente. Además, las posibilidades de automatización eran escasas.

Debido a todos estos problemas, BDD propone una evolución de las historias de usuario que permite, además de dejar más claros los casos y las condiciones para crear las pruebas, permitía su automatización. La estructura que se define es la siguiente:

Feature: [Nombre de la historia de usuario asociada]

Scenario: [Nombre del tipo de caso que queremos probar si OK o KO]

Given [un contexto del sistema, o un prerequisite como se definen en los Casos de uso tradicionales]

When [Ocurre un evento del sistema, dados unos condicionantes]

Then [establece el cambio de estado del sistema, tanto por parte del usuario como del sistema]

En el caso del cajero, el ejemplo sería:

Feature: Cliente quiere sacar dinero

Escenario: Existe dinero en la cuenta.

Dado Que en la cuenta corriente hay dinero

AND que la tarjeta es válida

AND que el cajero tiene billetes suficientes

Cuando El usuario ha introducido el dinero que quiere y le da a Continuar

Entonces Se descuenta lo extraído de la cuenta corriente

AND se dan los billetes pertinentes

AND se devuelve la tarjeta.

De esta forma se pueden diseñar las pruebas previamente a ser codificado nada. Como el objetivo siempre va a ser automatizar las pruebas, se buscó que se pudieran expresar en un lenguaje que sea entendible por la máquina y por el usuario de negocio. Este lenguaje se llama Gherkin.

Si nos damos cuenta, las *features*, se pueden definir de tal manera que sean el nombre de un test en el modo que Dan North propone para las pruebas.

Así de tal manera, que todo lo que hay después de las palabras clave, son nombres de prueba automatizables que se ejecutarían guiados por las *features* que el tester y el usuario de negocio han diseñado.

Es importante pararse a analizar esta idea: Imaginemos una aplicación que tenga 100 historias de usuario, cada ella con una media 4 escenario de prueba y cada escenario de prueba tenga a su vez 4 condiciones AND y que cada condición AND es un método de prueba. Esto nos daría potenciales: 800 casos de prueba.

Gracias a definir los escenarios, podemos reutilizar una gran parte de condicionantes, de manera automática, dejando documentado tanto para los usuarios de negocio como para los desarrolladores.

Además, herramientas de integración continua como Jenkins me permiten que, cada vez que haya una actualización de código, se comprueben que todas las pruebas están correctas y que el comportamiento sea el esperado.

4.4 Conclusiones

Las metodologías ágiles han permitido poner el foco en la automatización de las pruebas y en dar más énfasis en la colaboración de todos los perfiles más que en el proceso detallado.

Las técnicas de automatización principales y su significado que se han generado en la comunidad de pruebas ágiles son las siguientes:

1. Automatiza las pruebas para generar mejor código o TDD
2. Intenta establecer una definición de código correcto para el usuario o ATDD
3. Para probar la funcionalidad, describe el comportamiento de una manera reglada y automatizada o BDD

5 Herramientas de pruebas

En el siguiente apartado, comentaremos herramientas útiles a la hora de realizar pruebas.

5.1 Justificación a la necesidad de la automatización de pruebas

Hasta la proliferación de las técnicas de metodologías ágiles, toda la gestión del ciclo de vida, tanto de despliegues, pero también de pruebas, se hacía de manera manual. Esto implicaba las siguientes deficiencias:

- Mayor cantidad de errores humanos.
- Generación de mayor documentación, lo que quitaba de trabajo efectivo.
- Mucho mayor trabajo manual que, al ser tareas repetitivas, siempre un ordenador ganará en rapidez y coste al trabajo hecho por un humano.

Es por ello por lo que, poco a poco, las herramientas que se dedicaban única y exclusivamente al diseño y ejecución manual de las pruebas han introducido herramientas para su automatización. También la comunidad de software libre, en conjunción con el movimiento ágil, han desarrollado herramientas y metodologías orientadas a la automatización de las tareas.

A continuación, se va a realizar un pequeño desglose de herramientas útiles a la hora de la automatización de las pruebas, las métricas de calidad y el diseño y gestión del ciclo de vida de las pruebas.

5.2 Herramientas de diseño y gestión de pruebas

Mantis: es una aplicación escrita en Java que nos permite realizar el control de las incidencias de pruebas detectadas. Permite realizar filtros, controlar si se están arreglando o no, además de asignar los errores tanto a un desarrollador como verificar que las pruebas se han resuelto. Históricamente se ha integrado con Testlink. Es *open source*

Testlink: esta herramienta permite diseñar los casos de test, definiendo cuáles son necesarios realizarlos paso a paso, y asignarlos a los probadores. Se integra con Mantis y también es Open source

HP ALM: Antiguamente llamada Quality Center, ahora propiedad de HP. Es una herramienta que permite las funcionalidades de Mantis y Testlink juntas, además de permitir la automatización y la monitorización de las pruebas

Se pueden ver más herramientas en las referencias bibliográficas.

5.3 Herramientas de automatización de pruebas y métricas de calidad

SonarQube: es una herramienta que permite realizar análisis estáticos del código y de caja blanca, principalmente. Permite recoger métricas de calidad del código fuente escrito, como pueden ser las siguientes:

1. Líneas de código, clases.
2. Porcentaje de comentarios por líneas de código.
3. Complejidad de los métodos y las clases
4. Cobertura de código de las pruebas automatizadas
5. Cohesión de métodos y clases.
6. Prácticas de desarrollo inapropiadas (ej.: uso de números mágicos)
7. Código duplicado.

Es importante destacar que, como medidas de calidad, las Administraciones Públicas pueden utilizar las métricas que proporcionan las herramientas como SonarQube, u otra cualquiera, como criterio de aceptación a la hora de aceptar o no un código que un proveedor haya creado.

Una alternativa de pago a SonarQube, podría ser **Cast**, que incorpora más métricas de calidad que SonarQube.

SonarQube se integra con Jenkins, que pasamos a explicar a continuación.

Jenkins: es una herramienta de integración continua que, junto con repositorios compartidos como SVN, Git o Archiva, permite realizar una serie de tareas antes de integrar:

Ejecuta todas las pruebas que tengan asignadas, ya sean unitarias, de integración, de Interfaz de usuario. Para ello, utilizará cualquier tecnología compatible, como puede ser Cucumber para las pruebas basadas en BDD (que recuerdo que utilizan lenguaje Gherkin) o Selenium para las pruebas de interfaz de usuario.

Una vez que todas las pruebas están OK, Jenkins creará una nueva versión del código y lo subirá al repositorio que tenga definido. Si fallase algún proceso lo notificaría a la gente que tuviera descrita para que se resolviera, lo que debería ser prioridad en la ejecución.

Esta herramienta es la base de cualquier práctica ágil, ya que permite acortar los ciclos de desarrollo y permite automatizar cualquier tipo de pruebas que se quieran.

Una alternativa a Jenkins, que también tiene su versión .Net es **CruiseControl**, que también es *open source*.

Selenium: es una herramienta que nos permite automatizar todos los test orientados a las acciones de usuario. Típicamente, los usuarios utilizan los navegadores y a través de los navegadores pueden realizar una serie de acciones con el ratón y el teclado. En función de esas acciones la respuesta del programa ha de ser una u otra. Por ejemplo, introducir los datos en un formulario y darle a aceptar, o por ejemplo pinchar y arrastrar, son elementos que se pueden hacer en una aplicación web sobre navegador y que Selenium puede automatizar.

Para automatizar se puede utilizar dos maneras: grabar las acciones de usuario directamente en el navegador, o programar las acciones en código, con una serie de librería especiales que provee Selenium.

Cucumber: Es una herramienta escrita en Ruby que permite la creación de las *features* que se crearon con BDD. Tiene implementaciones para casi todos los lenguajes y está pensado para ser usado como consola de comandos. Como todos los anteriores, se puede integrar con Jenkins.

UFT: Antiguamente llamada QTP, ahora propiedad de HP. Una herramienta muy potente donde puedes grabar script de pruebas y parametrizarlos. Los datos pueden inyectarse a través de una hoja Excel, lo cual facilita mucho a la hora de lanzar pruebas con datos diferentes. Una de las grandes ventajas de esta herramienta es la integración con la suite de HP. Los scripts que grabemos con esta herramienta pueden subirse al ALM y desde ahí ejecutarlos, mientras el propio script deja una evidencia en formato pdf. Otra herramienta con la que se puede enlazar es con LoadRunner. Una herramienta de estrés de la aplicación en la cual se pueden grabar script o utilizar los grabados con el UFT.

Otras herramientas para realizar pruebas son: **SoapUI** y **Mock TestClient**, encargadas de ayudar en el proceso de pruebas de servicios web.

5.4 Conclusiones

Debido a mejoras de coste, reducción de errores y de tiempo; se hace totalmente necesario el uso de herramientas para la automatización de las pruebas.

Existen herramientas que mejoran la productividad, documentan los procesos y quitan trabajos repetitivos para permitir la automatización.

Por el lado de la gestión del proceso de pruebas y el diseño de los documentos de pruebas tenemos herramientas de software libre como Mantis y testlink y por el lado de pago, tenemos herramientas como HP ALM o UFT.

Además, para la automatización de las pruebas, más orientados metodologías o frameworks ágiles, tenemos SonarQube para la generación de métricas de calidad del código y Jenkins, como herramienta base para la integración continua donde aplicaciones como Cucumber o Selenium pueden ser ejecutados y, si se considera, aceptados para promocionar de entorno hasta llegar a producción.

6 Glosario

Error: Desviación entre el componente real y el componente esperado.

Defecto: Anomalía que da lugar a que no se lleve a cabo correctamente una función.

Fallo: Manifestación de un defecto.

Equivocación: Fallo humano.

Depuración: Localización y corrección de errores internos.

Prueba: Búsqueda de fallos para demostrar el defecto.

Pruebas de SW: Ejecución de un objeto de prueba.

Caso de prueba: Unión de una prueba y unas condiciones de entorno establecidas. [Precondiciones, entradas, salidas, postcondiciones].

Procedimiento de prueba: Pasos del caso de prueba.

Explosión de casos de prueba: El número de posibles casos de prueba.

Deuda técnica: se refiere a defectos del software que, cuanto más tardan en arreglarse, mayor será el coste del mismo, ya que solucionarlos en etapas posteriores es mucho más costoso que en primeras etapas del desarrollo.

Objeto de prueba: elemento del que se va verificar la calidad. Normalmente código o documentación.

RESUMEN ESQUEMÁTICO

1. La calidad está asociada a la ausencia de fallos, a la satisfacción del usuario y a la conformidad con los requerimientos
 - La ISO/IEC 25010:2011 realiza una definición de atributos de la calidad, que una vez valorados de forma independiente permiten evaluar la calidad de cualquier elemento. Estos atributos son la funcionalidad, portabilidad, fiabilidad, usabilidad, eficiencia y mantenibilidad.
 - La falta de calidad es crítica para un proyecto SW. Cuanto más avanza el ciclo de vida del software, el coste de la detección de un error sube de manera exponencial con un índice del exponente de 5.
 - Los estándares de “iure” más importantes son la familia de ISO/IEC 25000 y la ISO/IEC/IEEE 29119.
 - El estándar de “facto” más importante es el de la asociación sin ánimo de lucro ISTQB.
2. Lo principios de las pruebas software son:
 - La imposibilidad de demostrar la ausencia de defecto.
 - La ausencia de fallos no prueba que el software sea correcto.
 - Las pruebas exhaustivas no son posibles.
 - Se han de realizar pruebas tempranas.
 - Principio de localización en las pruebas.
 - La paradoja del pesticida.
 - Las pruebas dependen del contexto.
 - La calidad no es solo la ausencia de errores.
3. El ciclo de vida de pruebas genérico es el siguiente:
 - Planificación y control.
 - Análisis y diseño.
 - Implementación y ejecución.
 - Evolución y documentación.
4. Los perfiles más importantes a la hora de realizar las pruebas son:
 - El Responsable de Pruebas
 - El Probador.
 - A su vez estos roles se pueden subdividir en:
 - Analista.
 - Diseñador.
 - Automatizador.
 - Administrador de sistemas de pruebas.
5. Para realizar la planificación se crea un documento plan maestro de pruebas. Una posible estructura del mismo está en la norma IEEE 829. Es un documento vivo que va cambiando con el tiempo.
6. Los puntos más importantes para una planificación son:
 - el alcance de las pruebas y los riesgos,
 - establecer la estrategia de pruebas,
 - integrar y coordinar las actividades dentro del ciclo de vida software,
 - establecer recursos, roles y tareas,
 - seleccionar métricas de monitorización y avance.
7. Para la estimación de las pruebas existen dos aproximaciones:
 - El enfoque basado en métricas
 - El enfoque basado en experiencia.

8. La estimación de pruebas se deberá realizar de manera conjunta con una estrategia de pruebas, cuyos enfoques puede ser por un lado predictivos o reactivos, en función del momento del ciclo de vida.
9. Adicionalmente, para definir qué partes del software probar, los enfoques pueden ser:
 - Analíticos.
 - Basados en modelos.
 - Metódicos.
 - Basados en estándares.
 - Dinámicos o heurísticos.
 - De consulta.
 - Centrados en la regresión.
10. Existen muchos tipos de riesgos, pero los más importantes y que han de estar en la planificación son los de software.
11. En el ciclo de vida de pruebas es necesario que se defina el ciclo de vida de las incidencias detectadas, al igual que los medios de comunicación, priorización y resolución de las mismas.
12. En función del momento del desarrollo en que se realicen las pruebas, el tipo de prueba y la profundidad de la misma varía. Al igual que la evaluación de las pruebas funcionales o no funcionales.
13. Existen diversas técnicas para la definición de las pruebas, ya sea para el análisis de la calidad del código o técnicas estáticas o para la verificación de la funcionalidad, o técnicas dinámicas.
14. Dentro de las técnicas estáticas tenemos las revisiones, que pueden ser: Inspecciones, Walkthrough, Revisiones técnicas o revisiones informales. Por otro lado, tenemos el análisis de la calidad del código, la complejidad ciclomática, análisis de código duplicado y los ciclos entre paquetes.
15. Dentro de las técnicas dinámica o técnicas de diseño de pruebas tenemos las de caja negra o funciona o las de caja blanca o estructurales.
16. Las de caja negra establecen formas de analizar una aplicación para diseñar las pruebas. Estas técnicas son: clases de equivalencia, análisis de valores límite, tabla de decisión, pruebas de estado y pruebas de casos de uso. Estas técnicas suelen ser usadas para establecer los criterios de aceptación por parte del usuario.
17. Las técnicas de caja blanca nos permiten analizar la cantidad de código que vamos a analizar por cada prueba. El objetivo será maximizar el código cubierto. Las técnicas son: la cobertura de sentencias, la cobertura de ramas, la cobertura de condiciones y la cobertura de rutas.
18. Las perspectivas ágiles del testing inciden sobre técnicas, más que sobre metodologías.
19. Para implantar un framework de pruebas ágiles hay que tener en cuenta cuatro puntos, a saber:
 - Los testers son parte del equipo.
 - Mentalidad orientada al desarrollo ágil.
 - Automatización de pruebas.
 - Visión amplia de las mismas.
20. Las técnicas ágiles principales son: TDD, ATDD y BDD. BDD modifica la forma de trabajar al incluir su propia forma de historias de usuario, pero orientándolas al testing.
21. La automatización de las pruebas son una necesidad que nos provee de las siguientes ventajas:
 - Reducción de los errores humanos.
 - Reducción de costes de pruebas.
 - Reducción del tiempo dedicado a pruebas.
22. La gestión de datos de las pruebas es un reto, sobre todo por la diversidad de entornos y de versiones del sistema.
23. Las herramientas de pruebas se pueden clasificar en dos grandes grupos:
 - Herramientas de automatización y métricas de calidad (Ej: Jenkins, SonarQube)

- Herramientas de diseño y gestión de pruebas. (Ej: HP ALM, UFT , Mantis)

PREGUNTAS DE TEST

- 1) ¿Cuáles de los siguientes atributos de calidad NO está definido en la ISO/IEC 25010?
 - a) Portabilidad
 - b) Mantenibilidad
 - c) Accesibilidad
 - d) Funcionalidad
- 2)Cuál de las siguientes afirmaciones es cierta:
 - a) Actualmente los estándares de testing más importantes los realiza ISO, que es la organización internacional de estandarización más importante.
 - b) Intuitivamente es fácil explicar que incorporando calidad a los desarrollos software, se mejora los plazos y la satisfacción del cliente.
 - c) La calidad del software se refiere exclusivamente a la validación de requisitos ya previamente definidos en una norma
 - d) Ninguna de las anteriores es cierta
- 3)Cuál de los siguientes NO forma parte de la de la ISO/IEC/IEEE 29119
 - a) ISO/IEC/IEEE 29119-4 Testing basado en palabras clave
 - b) ISO/IEC/IEEE 29119-1 Conceptos y definiciones
 - c) ISO/IEC/IEEE 29119-2 Proceso de pruebas
 - d) Todas las anteriores forman parte del estándar.
- 4) Respecto de los principios de las pruebas software que define el ISTQB, que es el estándar de facto más seguido en la industria:
 - a) Se define como importante que la calidad sea exclusivamente la búsqueda de la ausencia de errores
 - b) Identifica como objetivo ideal realizar pruebas exhaustivas y profundas.
 - c) Tras varios ciclos de pruebas, si se siguen realizando las mismas pruebas, estas dejarán de ser efectivas
 - d) Resalta que el objetivo final es la demostración de la correctitud del software
- 5) Respecto a los roles típicos que existen en la gestión de las pruebas, di cual es cierta:
 - a) Los perfiles básicos son el responsable de pruebas, el probador y el diseñador de pruebas
 - b) El automatizador es aquel que verifica las posibilidades de automatización de las pruebas y lo lleva a cabo si así se considera
 - c) Cada perfil ha de ser realizado solo por una única persona, siendo imposible que una persona realice dos roles en distintas fases del proyecto.
 - d) El automatizador realiza ejecuciones a mano del diseño de pruebas establecido, siendo este su principal cometido
- 6)Cuál de los siguientes epígrafes NO forma parte del plan maestro de pruebas según establece la norma IEEE 829 "Standard for test Documentation"
 - a) Riesgos
 - b) Criterios de interrupción y rechazo

- c) Características a probar
 - d) Entregables
- 7) A la hora de definir la estrategia de pruebas, ¿Cuál de los siguientes no es un enfoque de pruebas, según el ISTQB?
- a) Enfoque analítico
 - b) Enfoque centrado en regresión
 - c) Enfoque reactivo
 - d) Todas las anteriores son correctas
- 8) A la hora de realizar pruebas funcionales, di cual es la correcta:
- a) Se ha de evaluar la precisión de los resultados.
 - b) La seguridad, al ser un requisito no funcional, no ha de ser probado en las pruebas funcionales
 - c) La interoperabilidad no es criterio para evaluar en las pruebas funcionales.
 - d) Las pruebas funcionales son una parte importante de los criterios de aceptación del usuario.
- 9) Según define el estándar IEEE 1028,
- a) Las inspecciones son la forma menos formal de revisiones
 - b) Las revisiones informales suelen ser iniciadas por el responsable de pruebas, y a pesar de su nombre, es necesaria cierta documentación
 - c) En los walkthrough se distribuye con antelación la documentación, pero el autor de las pruebas y el moderador son necesariamente una persona distinta
 - d) Ninguna de las anteriores
- 10) Sobre las pruebas software, di cual es cierta:
- a) La complejidad ciclomática es una técnica estática
 - b) Las técnicas dinámicas son la base para diseñar las pruebas de aceptación del usuario
 - c) Las pruebas de caja negra ayudan a evitar la explosión combinatoria y a realizar pruebas más eficientes.
 - d) Todas las anteriores son ciertas
- 11) Según Brian Marick, ¿Cuál de los siguientes NO es un factor de éxito para la implantación de pruebas ágiles?
- a) A pesar de estar en equipos ágiles, se le ha de dar más peso a la documentación y al proceso que a las personas
 - b) Se han de integrar a los “testers” en el equipo.
 - c) Automatizar es importante, pero siempre enfocado a obtener una retroalimentación rápida y reducir al máximo posible la deuda técnica
 - d) Todas las anteriores son falsas.
- 12) ¿Quién hablo por primera vez de TDD en 2003?
- a) Dan North
 - b) Kent Beck
 - c) Kevin Nitmick
 - d) Noob Saibot
- 13) Respecto de las “feature” que define BDD:

- a) El objetivo va a ser siempre probar todo lo posible con un lenguaje solo apto para las máquinas
- b) Se podría decir que el campo Dado son las precondiciones, el Cuándo es la acción realizada por el usuario y el Entonces son las postcondiciones.
- c) El lenguaje de intercambio diseñado por BDD se llama Cucumber.
- d) En los apartados Dado y Escenario es donde se pueden usar las palabras clave And y OR

14) Respecto a las herramientas de automatización

- a) Se pueden realizar una automatización de pruebas sin usar Jenkins.
- b) SonarQube, antiguamente Sonar, es una herramienta que permite obtener métricas del código.
- c) Selenium permite automatizar pruebas utilizando el mismo interfaz que el usuario.
- d) Todas las anteriores son ciertas

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	c
2	d
3	a
4	c
5	b
6	b
7	d
8	a
9	d
10	d
11	a
12	b
13	b
14	d

BIBLIOGRAFÍA BÁSICA

1. www.iso.org
2. www.ieee.org
3. "A practitioners Guide to Software Test Design", Copeland
4. "Foundations of Software Testing", Graham, Veenendaal, Evans, Black.
5. Aplicación Google Play TPI Next, TPI
6. "Software estimation: demystifying the Black Art", Steve McConnell
7. "Test Driven Development by example", Kent Beck
8. "Introducing BDD", Dan North

BIBLIOGRAFÍA PARA AMPLIAR EL TEMA

9. http://qatestingtools.com/testing-tool-article/best_automation_testing_tools_2017
10. <https://www.infoq.com/news/2010/04/dont-automate-acceptance-tests>
11. <http://www.softwaretestinghelp.com/15-best-test-management-tools-for-software-testers/>
12. <https://www.tmmi.org/>
13. <http://in2test.lsi.uniovi.es/qt26/>
14. <http://context-driven-testing.com/>
15. <http://www.softwaretestingstandard.org/29119petitionresponse.php>